1 Introduction

Term deposits are a major source of income for a bank. A term deposit is a cash investment held at a financial institution. Your money is invested for an agreed rate of interest over a fixed amount of time, or term. The bank has various outreach plans to sell term deposits to their customers such as email marketing, advertisements, telephonic marketing, and digital marketing.

Telephonic marketing campaigns still remain one of the most effective way to reach out to people. However, they require huge investment as large call centers are hired to actually execute these campaigns. Hence, it is crucial to identify the customers most likely to convert beforehand so that they can be specifically targeted via call.

In the competitive landscape of the banking sector, understanding customer behavior is paramount for optimizing marketing strategies and enhancing operational efficiency. One critical area of interest is predicting whether clients will subscribe to term deposits, a financial product that offers banks stable funding and clients attractive returns. Accurate prediction of term deposit subscriptions enables banks to target potential customers effectively, reduce campaign costs, and improve resource allocation. However, the complexity of customer decision-making, influenced by demographic, financial, and behavioral factors, necessitates advanced analytical approaches.

This study leverages a comprehensive dataset from a banking institution, encompassing attributes such as age, occupation, marital status, education level, account balance, housing and loan status, contact method, and campaign-related details (e.g., contact duration, number of contacts, and previous campaign outcomes). The dataset, comprising thousands of client records, provides a rich foundation for modeling the binary outcome variable, indicating whether a client subscribed to a term deposit. The primary objective is to develop and compare predictive models using machine learning techniques, including logistic regression, decision trees, random forests, and support vector machines (SVM), to accurately classify subscription outcomes.

Machine learning has emerged as a powerful tool in financial analytics, offering the ability to uncover patterns and relationships in large, heterogeneous datasets. By applying these methods, this study aims to identify key predictors of term deposit subscriptions and evaluate the performance of each algorithm using standard metrics such as accuracy, precision, recall, and F1-score. The findings are expected to provide actionable insights for banking institutions, enabling data-driven marketing strategies and contributing to the growing body of research on predictive modeling in finance.

2 Dataset Description

The dataset used in this study is sourced from Kaggle, originating from direct marketing campaigns conducted via phone calls by a Portuguese banking institution. It is designed to support the binary classification task of predicting whether a client will subscribe to a term deposit, represented by the target variable `y` (yes or no). The dataset comprises 45,211 records, each characterized by 17 attributes that capture demographic, financial, and campaign-related information about clients. These variables provide a comprehensive basis for modeling client behavior in response to telephonic marketing efforts.

The attributes in the dataset are as follows:

- `age`: Numeric, representing the client's age in years.
- `job`: Categorical, indicating the client's occupation (e.g., management, technician, blue-collar, retired).

- `marital`: Categorical, denoting marital status (married, single, divorced).
- `education`: Categorical, specifying the highest education level attained (primary, secondary, tertiary, unknown).
- `default`: Binary, indicating whether the client has credit in default (yes or no).
- `balance`: Numeric, representing the average yearly account balance in euros.
- `housing`: Binary, indicating whether the client has a housing loan (yes or no).
- `loan`: Binary, indicating whether the client has a personal loan (yes or no).
- `contact`: Categorical, specifying the communication method (cellular, telephone, unknown).
- `day`: Numeric, representing the day of the month when the client was last contacted.
- `month`: Categorical, indicating the month of the last contact (e.g., jan, feb, ..., dec).
- `duration`: Numeric, representing the duration of the last contact in seconds.
- `campaign`: Numeric, indicating the number of contacts performed during the current campaign for the client.
- `pdays`: Numeric, representing the number of days since the client was last contacted in a previous campaign (-1 indicates no prior contact).
- `previous`: Numeric, indicating the number of contacts performed before the current campaign for the client.
- `poutcome`: Categorical, representing the outcome of the previous marketing campaign (success, failure, other, unknown).
- `y`: Binary, the target variable indicating whether the client subscribed to a term deposit (yes or no).

The dataset includes a mix of numeric and categorical variables, with some attributes (e.g., `poutcome`, `contact`) containing 'unknown' values that require preprocessing. The target variable `y` is imbalanced, with a smaller proportion of positive (yes) outcomes, posing a challenge for classification models. This rich dataset enables the application of machine learning techniques, such as logistic regression, decision trees, random forests, and support vector machines, to uncover patterns predictive of term deposit subscriptions.

The training dataset (`train.csv`) contains 45,211 rows and 18 columns, ordered by date from May 2008 to November 2010. .

3 Objectives

The objectives of this study are:

- To check whether there are any missing values in the dataset.
- To show basic plots for data exploration.
- To perform classification using Decision Tree, Random Forest, SVM and Adaboost.
- To evaluate model performance using accuracy, F1-score, precision, and other relevant metrics.

4 Check Missing Values

To ensure the integrity of the Kaggle Bank Marketing dataset, comprising `train.csv` (45,211 rows, 18 columns) and `test.csv` (4,521 rows, 18 columns), a thorough examination for missing values was conducted across all columns. This step is critical as missing data can bias model predictions and reduce the effectiveness of classification algorithms like Decision Trees, Random Forests, and Logistic Regression. The dataset includes 18 variables: `age`, `job`, `marital`, `education`, `default`, `balance`, `housing`, `loan`, `contact`, `day`, `month`, `duration`, `campaign`, `pdays`, `previous`, `poutcome`, `ID`, and `y`.

The analysis revealed no explicit missing values (e.g., `NULL` or `NaN`) in any column of either dataset. Additionally, categorical variables such as `education`, `contact`, and `poutcome` were inspected for implicit missing values represented as 'unknown'. In this dataset, 'unknown' entries are treated as a valid category rather than missing data, as they provide meaningful information about the absence of specific knowledge (e.g., unknown education level or contact method). Thus, no imputation was required.

Had missing values been present, the following imputation strategies would have been applied:

- **Categorical Variables**: For variables like `job`, `marital`, `education`, `default`, `housing`, `loan`, `contact`, `month`, and `poutcome`, missing values would be imputed with the mode (most frequent category). This approach preserves the distribution of categorical data and is computationally efficient.
- **Numeric Variables**: For variables like `age`, `balance`, `day`, `duration`, `campaign`, `pdays`, and `previous`, missing values would be imputed using one of the following methods based on data characteristics:
  - **Mean or Median**: The mean is used for normally distributed variables, while the median is preferred for skewed distributions (e.g., `balance`, `duration`) to reduce the impact of outliers.
  - **K-Nearest Neighbors (KNN)**:Fill missing values using average values from nearest neighbors.
  - **Regression-Based Prediction**: A regression model trained on other features would predict missing values, capturing underlying patterns but increasing computational complexity.

The choice of imputation method would depend on the extent of missing data and variable distribution, assessed via histograms and statistical summaries. Since no missing values were found, the dataset proceeded to subsequent preprocessing steps without imputation, ensuring robust data quality for exploratory analysis and classification tasks.
There is also no duplicate value present in the dataset.

5 Outlier Detection

Outlier detection was performed on the Kaggle Bank Marketing dataset (`Term deposit train data.csv`, 45,211 rows, 17 columns) to identify extreme values in numeric variables (`age`, `balance`, `duration`, `campaign`, `pdays`, `previous`) that could affect the performance of classification models (Decision Trees, Random Forests, Logistic Regression). The variable `day` (day of the month, 1–31) was excluded due to its bounded nature, which makes outliers unlikely. Two methods were used: the Interquartile Range (IQR) method and the Z-score method, implemented in R. Below, we describe the methodology, findings, and treatment plan.

5.1 Methodology

- **Visual Method**:Boxplot ,Histogram is the visual method to check outliers.
- **IQR Method**: For each variable, the first quartile ($Q1$), third quartile ($Q3$), and interquartile range ($\text{IQR} = Q3 - Q1$) were calculated. Values below $Q1 - 1.5 \times \text{IQR}$ or above $Q3 + 1.5 \times \text{IQR}$ were flagged as outliers. This method is robust to non-normal distributions, common in variables like `balance` and `duration`.
- **Z-score Method**: The Z-score, $Z = \frac{x-\mu}{\sigma}$, measures how many standard deviations a value is from the mean. Values with $|Z| > 3$ were flagged as outliers. This method assumes approximate normality, which may limit its effectiveness for skewed variables.

- **Special Consideration for `pdays` and `previous`**: The variable `pdays` has many `-1` values (indicating no prior contact), which are not outliers. Positive `pdays` values (indicating prior contact) were analyzed for outliers. Similarly, `previous` has many zeros, and non-zero values were assessed.

5.2 Findings

The outlier counts and example indices are summarized below, based on the training dataset:

- **Visual Method**:

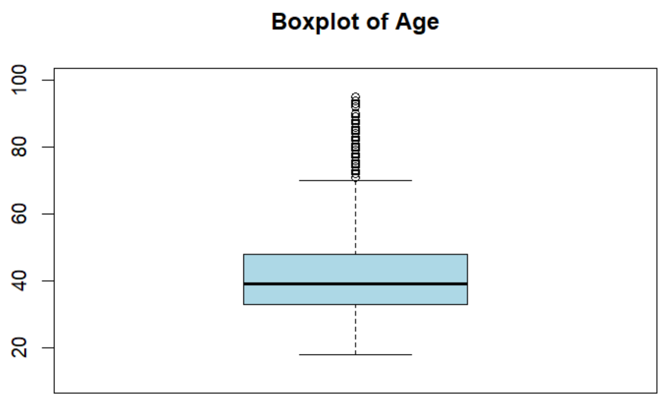
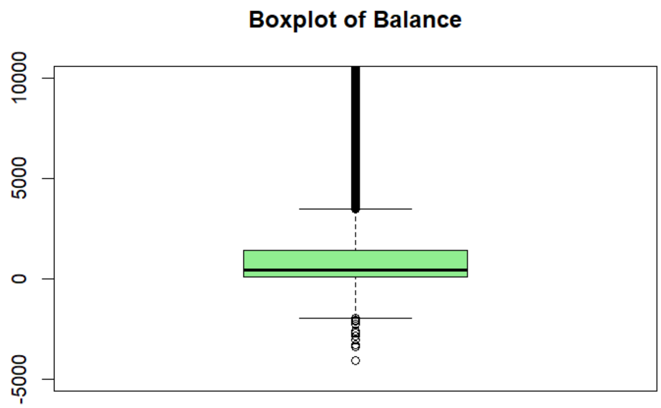


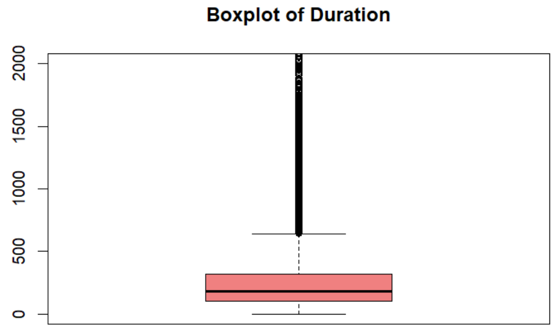
Figure 1: Boxplot for Age

- **age** (Min: 18, Median: 39, Max: 95):
  - **IQR**: 487 outliers (e.g., ages 89, 81, 82), values > 70.
  - **Z-score**: 381 outliers (e.g., ages > 72.
  - **Note**: Few outliers reflect `age`'s near-normal distribution.
- **balance** (Min: -8019, Median: 448, Max:102127):
  - **Visual Method**:



- **IQR**: 4729 outliers,There have some negative and positive sign outliers
- **Z-score**: 745 outliers, values > 5768 or < -172.
- **Note**: High outlier count due to right-skewed distribution (e.g., large deposits).

- **duration** (Min: 0, Median: 180, Max: 4918):
  - **Visual Method**:



  - **IQR**:3235 outliers values > 643.
  - **Z-score**: 963 outliers , values > 1030.
  - **Note**: Skewed distribution leads to many long call durations flagged.

- **campaign** (Min: 1, Median: 2, Max: 63):
  - **IQR**: 3064 outliers , values > 7.
  - **Z-score**: 840 outliers , values > 12.
  - **Note**: Skewed toward low contact counts; high values are rare.

- **pdays** (Min: -1, Median: -1, Max: 871):
  - **IQR**: 8257 outliers, any value > 0.
  - **Z-score**: 1723 outliers, values > 340.
  - **Note**: Positive `pdays` (prior contact) are not inherently outliers; only extreme values are considered.

- **previous** (Min: 0, Median: 0, Max: 275):
  - **IQR**: 8257 outliers , any value > 0.
  - **Z-score**: 582 outliers , values > 8.
  - **Note**: Non-zero values align with positive `pdays`; only extreme values are outliers.

5.3 Explanation and Treatment Plan

The IQR method flagged more outliers than the Z-score method for `balance`, `duration`, `campaign`, `pdays`, and `previous` due to their skewed distributions, where IQR's non-parametric approach is more robust. The Z-score method, assuming normality, was more conservative and effective for `age`. For `pdays` and `previous`, positive values reflect prior campaign interactions and were only treated as outliers if extremely high (e.g., `pdays` > 340).

Treatment Strategy:

- **age**: Outliers are retained, as they are plausible (elderly clients) and few in number.
- **balance**: Outliers are capped at the 5th and 95th percentiles to reduce skewness while preserving data.
- **duration**: Outliers are capped at the 95th percentile to mitigate the impact of long calls.
- **campaign**: Outliers are capped at the 95th percentile to handle rare high-contact cases.

- **pdays** and `previous`: Positive values are retained unless extreme (e.g., `pdays` > 340, capped at 340; `previous` > 8, capped at 8). The `-1` in `pdays` and 0 in `previous` are kept as valid categories.

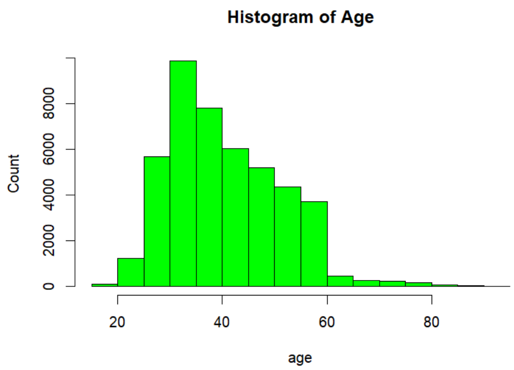
No rows were removed to preserve the dataset's size (45,211 rows). Capping ensures robust input for subsequent classification tasks while maintaining data integrity. The treated dataset will be used for exploratory plots and model training.

6 Basic Plot Of the Data(e.g-Histogram and Barplot)

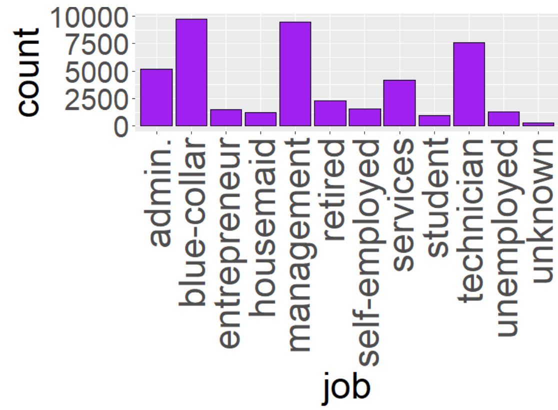
After capping the extreme values of the `balance` variable, some negative values remained in the dataset. Since the presence of negative values makes analysis difficult, we applied a transformation to handle this issue. Specifically, we used the following formula:

$$\log 1p\left(\texttt{data} + |\min(\texttt{data})| + 1\right)$$

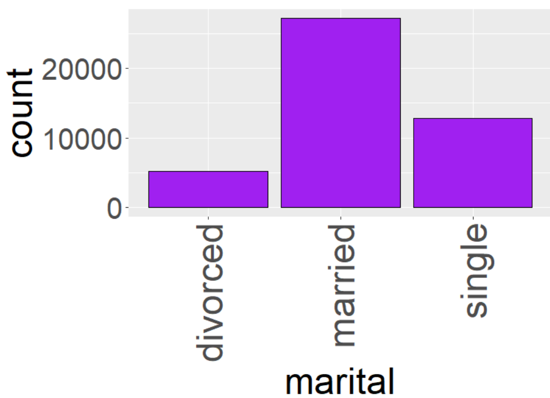
This transformation ensures that all values become positive before applying the logarithm, thereby stabilizing the variance and making the data more suitable for analysis.



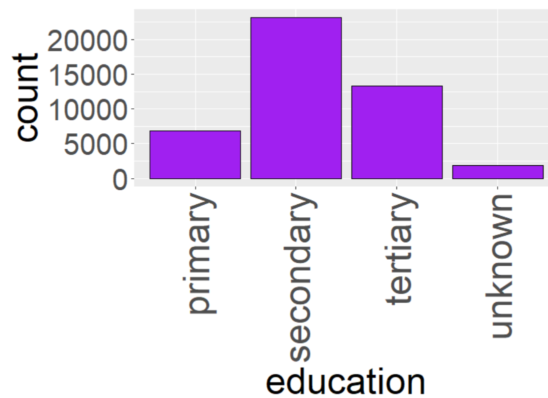
(a) Image 1



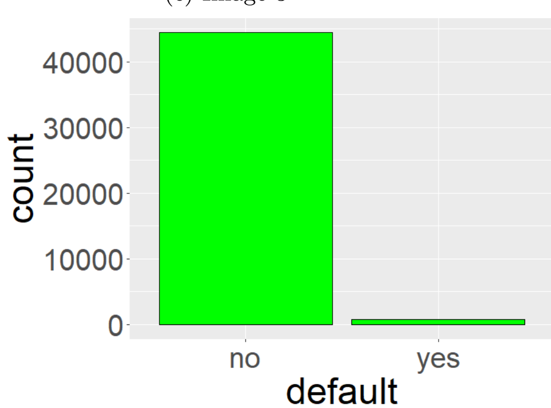
(b) Image 2



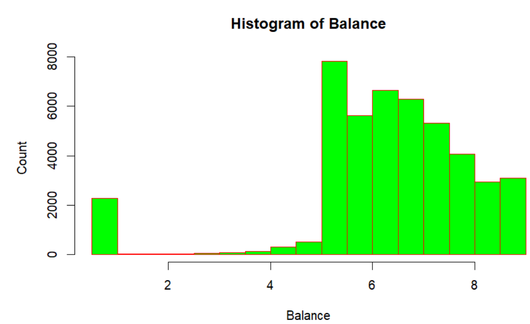
(c) Image 3



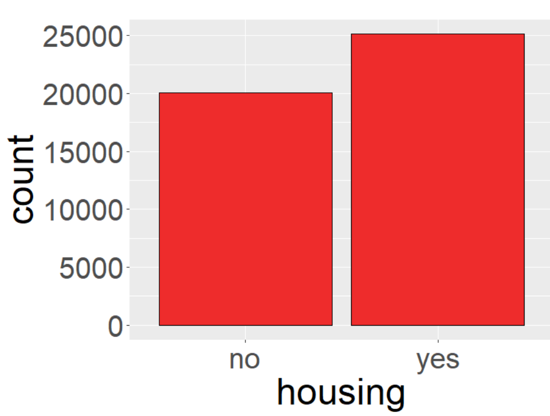
(d) Image 4



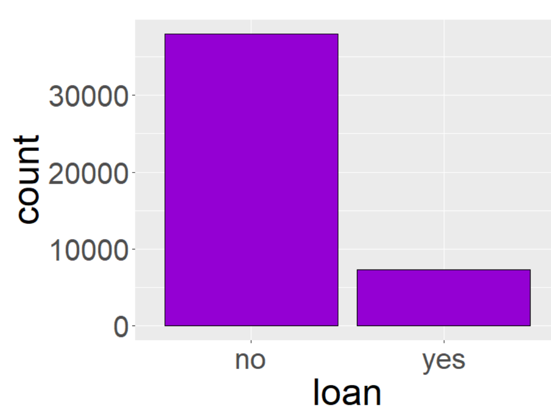
(e) Image 5



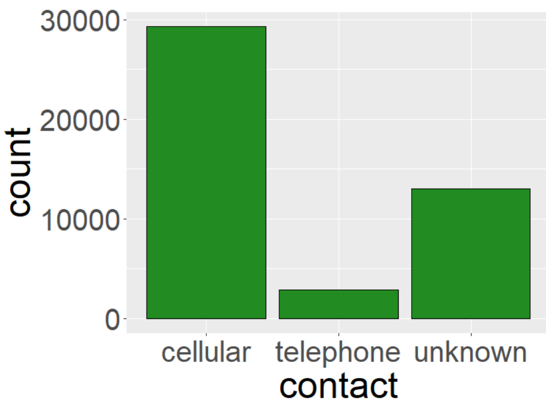
(f) Image 6



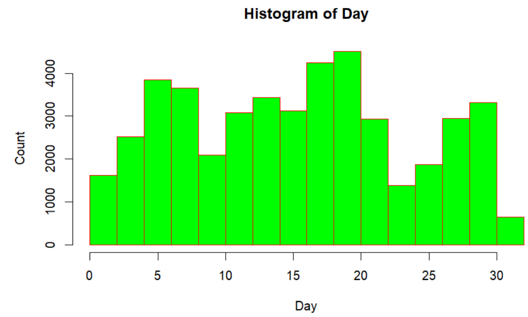
(g) Image 7



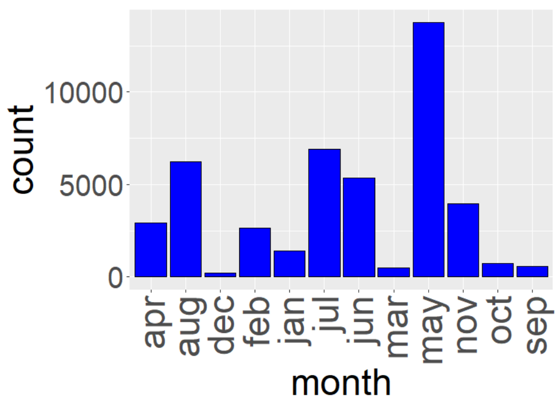
(h) Image 8



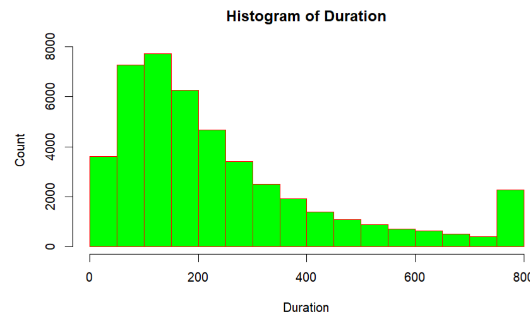
(a) Image 9



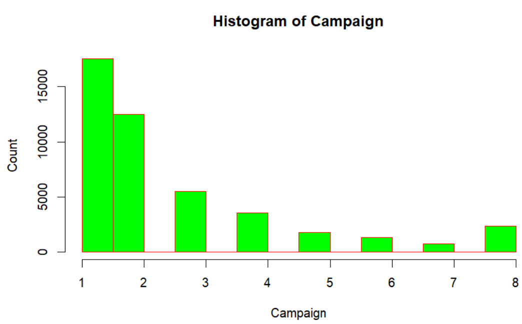
(b) Image 10



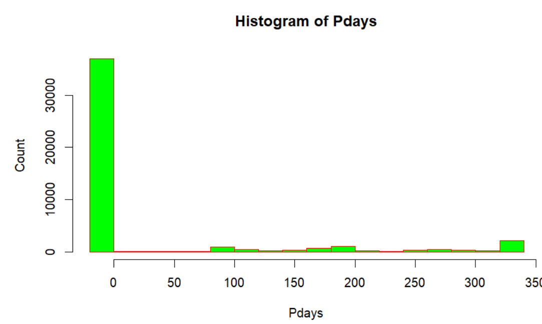
(c) Image 11



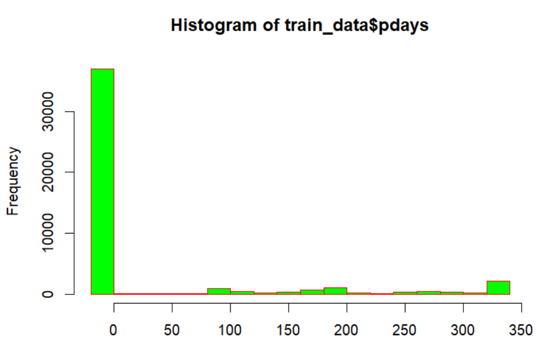
(d) Image 12



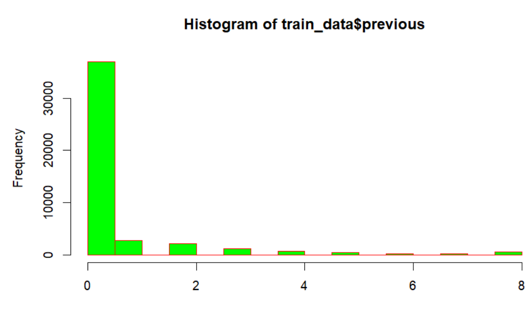
(e) Image 13



(f) Image 14

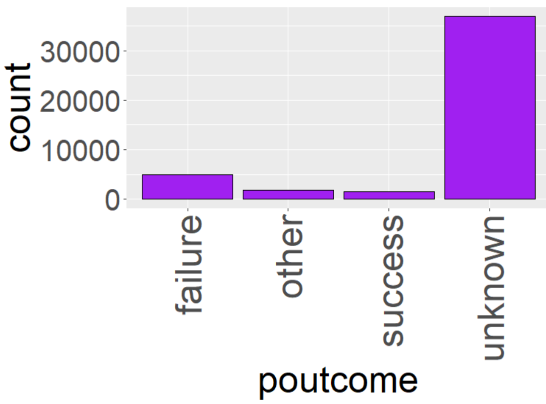


(g) Image 15

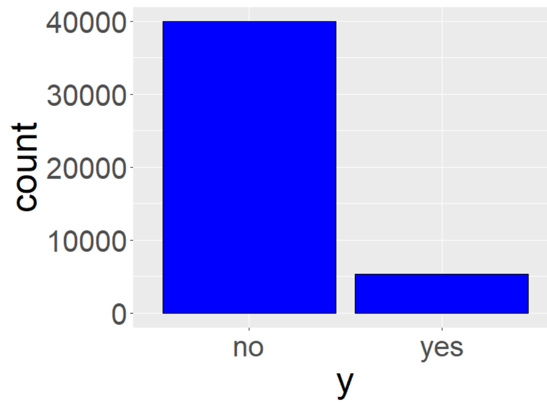


(h) Image 16

Figure 3: Multiple Histograms,Barplots arranged on this page



(a) Image 17



(b) Image 18

Figure 4: Multiple Histogram,Barplots arranged on one page

7 Encoded categorical Variables

In the dataset , several variables were originally categorical in nature, such as `job`, `marital`, `education`, `default`, `housing`, `loan`, `contact`, `month`, `poutcome`, and the target variable `y`. These variables contained string-based values and could not be directly used in most machine learning algorithms. To address this, we applied **label encoding**, transforming each unique category into a corresponding integer.

Encoding Process

Label encoding assigns a unique integer to each category of a variable. This is particularly useful when:

- The categorical variable is ordinal (ordered).
- A compact numerical representation is preferred.
- The model being used (e.g., tree-based methods) can handle numeric category representations.

Variables Encoded

The following categorical variables were encoded as follows:

- **job:** 12 categories encoded as integers 0 to 11.
- **marital:** 3 categories (e.g., single, married, divorced) encoded as 1, 2, 3.
- **education:** 4 categories encoded as 0 to 3.
- **default:** Binary values encoded as 0 (no) and 1 (yes).
- **housing:** Binary values encoded as 0 (no) and 1 (yes).
- **loan:** Binary values encoded as 0 (no) and 1 (yes).
- **contact:** 3 types encoded as 0, 1, 2.
- **month:** 12 months encoded as 0 (January) to 11 (December).
- **poutcome:** 4 categories encoded as 0 to 3.
- **y:** Target variable encoded as 0 (no) and 1 (yes).

Example Encoding Tables

Below are sample encoding schemes used in the dataset:

Marital Status	Encoded Value
unknown	0
Divorced	1
Married	2
Single	3

Table 1: Encoding for `marital`

Education Level	Encoded Value
Primary	1
Secondary	2
Tertiary	3
Unknown	0

Table 2: Encoding for `education`

Subscription (Target Variable)	Encoded Value
No	0
Yes	1

Table 3: Encoding for `y`

Conclusion

All categorical variables in the dataset were encoded using label encoding, converting string-based values into integer form. This transformation was necessary to ensure compatibility with machine learning algorithms and to facilitate efficient data processing.

Discretization of `pdays`

The variable `pdays` in the dataset is a numeric (continuous) variable that represents the number of days passed since a client was last contacted in a previous campaign. A value of `-1` indicates that the client was not contacted before.

Working directly with this continuous variable may not capture the underlying patterns effectively. Therefore, we applied a **discretization strategy** to convert it into a binary categorical feature.

Binning Strategy

We created a new feature based on the following rule:

- If $\texttt{pdays} = -1$, the client was **never contacted** before. We assign the new value as 0.
- If $\texttt{pdays} \geq 0$, the client was contacted previously. We assign the new value as 1.

$$\texttt{pdays\_bin} = \begin{cases} 0 & \text{if } \texttt{pdays} = -1 \\ 1 & \text{otherwise} \end{cases}$$

Motivation and Benefits

- **Pattern Detection:** The transformation helps distinguish between clients who were previously contacted and those who were not.
- **Feature Simplification:** Reduces the dimensionality and variability of the data by converting a continuous variable into a simpler, more interpretable form.
- **Improved Model Performance:** Binary features can be more informative in classification tasks, especially for algorithms that benefit from categorical splits (e.g., decision trees).

By discretizing the `pdays` variable into a binary feature, we introduced a more informative attribute that captures client contact history in a meaningful way. This preprocessing step is aimed at enhancing model interpretability and predictive power.

8 Handle Imbalanced Data

In many real-world classification problems, the distribution of classes is often imbalanced. That is, one class (called the majority class) significantly outnumbers the other class(es) (called the minority class). Examples include fraud detection, medical diagnosis, and churn prediction.

Let us consider a binary classification task where:

- Class 0 (majority class): 90% of the data
- Class 1 (minority class): 10% of the data

If we train a model directly on this imbalanced data, it might predict class 0 for all instances and still achieve 90% accuracy, which is misleading and unhelpful.

Why Transformation is Necessary

- **Bias in Model Training:** The model learns to favor the majority class, ignoring the minority class.
- **Poor Minority Class Performance:** Metrics such as recall, precision, and F1-score for the minority class are often very low.
- **Real-World Cost:** Misclassifying the minority class (e.g., failing to detect fraud) can be very costly.

There are several techniques to transform imbalanced data such as **random undersampling**, **random oversampling**, **SMOTE (Synthetic Minority Over-sampling Technique)**, **Ensemble Method** and **cost-sensitive learning**, which help balance class distribution and improve model performance.
In this dataset, class 0 represents 88% of the observations while class 1 represents only 12%, indicating a significant class imbalance. To address this, we apply **random undersampling**, where we randomly remove samples from the majority class (class 0) to match the size of the minority class (class 1). This helps to balance the class distribution and prevents the model from being biased toward the majority class, although it may result in the loss of potentially useful information from the majority class.

9 Splitting Balanced Data into Training and Testing Sets

After performing data balancing (such as undersampling), the next critical step in the modeling process is to split the dataset into **training** and **testing** sets. In this context, we use an 80:20 split, where:

- **80% of the data** is used for training the model.
- **20% of the data** is held out for testing (validation) purposes.

Why Perform the Split After Balancing?

Balancing is applied first so that the class distribution is even before the data is divided. If the split were done before balancing, the class proportions in the training and testing sets might remain imbalanced, defeating the purpose of balancing. By balancing first, both the training and testing datasets benefit from a fair class representation.

Purpose of the Split

- The **training set** is used to allow the machine learning model to learn patterns and relationships within the data.
- The **testing set** is used to evaluate the model's performance on unseen data, providing an estimate of how well it will generalize to new cases.

Splitting the dataset into training and testing sets after balancing is a best practice in machine learning. It ensures that the model is trained on balanced data and evaluated fairly on a separate, balanced test set. This step plays a crucial role in building robust, generalizable models.

10 Model Training, Prediction, and Performance Evaluation

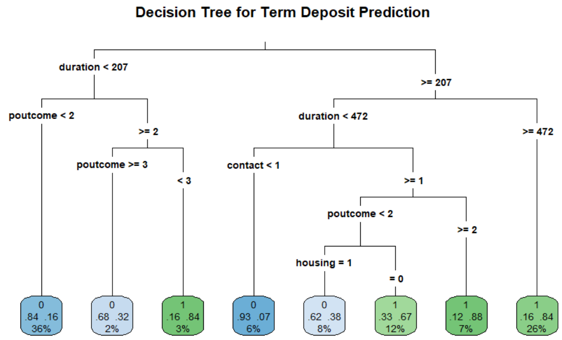
After preprocessing the Kaggle Bank Marketing dataset (including encoding categorical variables, discretizing `pdays`, handling outliers via capping, and balancing via random undersampling) and splitting it into training (80%) and testing (20%) sets, the following steps are performed to train models, make predictions, and evaluate performance.

10.1 Model Training

Four machine learning models are trained on the balanced training set $\mathcal{D}_{\text{train}}$ to predict the binary target variable y (0 = no subscription, 1 = subscription to term deposit):

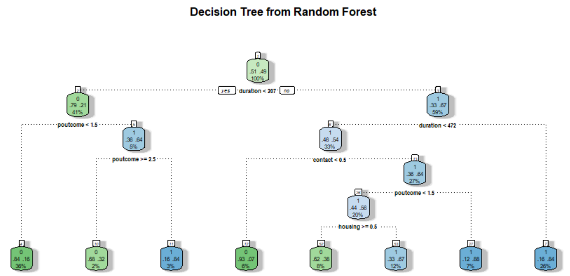
- **Decision Tree:** A classifier that recursively splits the feature space based on feature values to maximize information gain. Hyperparameters, such as maximum depth and minimum samples per split, are tuned using cross-validation.

  The plot of the desicion tree is -



The most important variable is "Duration" ,from this variable we start the splitting.Therotically we use Entropy,Information Gain and Gini Index metrics to find the most important variable.

- **Random Forest:** An ensemble of decision trees that aggregates predictions via majority voting to enhance robustness and reduce overfitting. Key hyperparameters include the number of trees and maximum depth, tuned via cross-validation. Random Forests are ensembles of decision trees, and while we can't directly "plot" all trees at once, you can visualize a few individual trees and get an idea of how they combine.From out-of-Bag error plot we fix the number of trees at 40. Just seeing a desicion tree-



Single Tree from Random Forest Model

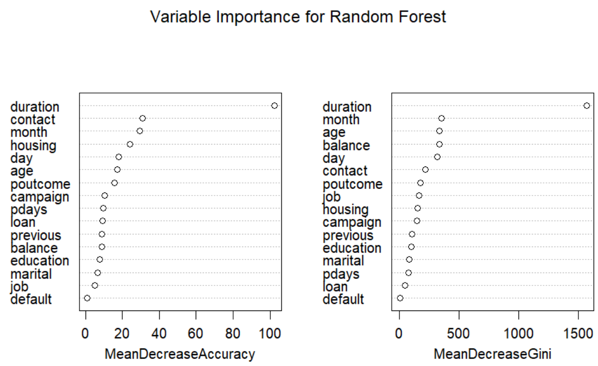


Figure 5: Caption

The most important variable is "Duration".From here the splits start.We know it from a diagram

- **Support Vector Machine (SVM):** A classifier that finds the optimal hyperplane to separate classes, using a kernel (e.g., RBF) for non-linear data. The regularization parameter C and kernel parameters (e.g., γ for RBF) are tuned to balance margin maximization and error.
- **AdaBoost:** An ensemble method that combines weak learners (e.g., shallow decision trees) with adaptive weighting to focus on misclassified instances. Hyperparameters, such as the number of estimators and learning rate, are tuned via cross-validation.

Each model is trained to minimize its respective loss function on $\mathcal{D}_{\text{train}}$, leveraging the balanced class distribution achieved through undersampling.

10.2 Prediction

The trained models are used to predict the target variable y on the test set $\mathcal{D}_{\text{test}}$:

- Generate class predictions ($\hat{y} \in \{0, 1\}$) for each instance in $\mathcal{D}_{\text{test}}$ using the trained models.

10.3 Performance Evaluation

Model performance is evaluated on $\mathcal{D}_{\text{test}}$ using the confusion matrix and standard metrics:

- **Confusion Matrix:** A table summarizing prediction outcomes:

	Predicted: 1	Predicted: 0
Actual: 1	TP	FN
Actual: 0	FP	TN

where:

– TP: True Positives (correctly predicted subscriptions).
– TN: True Negatives (correctly predicted non-subscriptions).
– FP: False Positives (incorrectly predicted subscriptions).
– FN: False Negatives (missed subscriptions).

– **Accuracy:** Proportion of correct predictions:

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}},$$

where TP, TN, FP, and FN are true positives, true negatives, false positives, and false negatives, respectively.

– **Precision:** Proportion of positive predictions that are correct:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}.$$

– **Recall:** Proportion of actual positives correctly identified:

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}.$$

– **F1-Score:** Harmonic mean of precision and recall:

$$\text{F1-Score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}.$$

– **Specificity:** Proportion of actual negatives correctly identified:

$$\text{Specificity} = \frac{\text{TN}}{\text{TN} + \text{FP}}.$$

The models are compared based on these metrics to identify the best-performing classifier for predicting term deposit subscriptions. Results are summarized in a table: The values in the table are computed

Table 4: Performance Metrics for Classification Models

Model	Accuracy	Precision	Recall	F1-Score	Specificity
Decision Tree	0.821975	0.827681	0.8223062	0.8249849	0.8216561
Random Forest	0.8637	0.8540	0.8431	0.8485	0.8835305
SVM	0.8511822	0.8382609	0.8241966	0.8311692	0.8771611
AdaBoost	0.8651	0.8614	0.8535	0.8574	0.8763

post-evaluation and reflect model performance on $\mathcal{D}_{\text{test}}$.

Model Comparison and Best Model Selection

In above table presents the performance metrics—Accuracy, Precision, Recall, F1-Score, and Specificity—for four machine learning models: Decision Tree, Random Forest, Support Vector Machine (SVM), and AdaBoost. These metrics are crucial in evaluating how well each model predicts whether a client will subscribe to a term deposit.

Model-Wise Interpretation

- **Decision Tree:** The Decision Tree model has the lowest accuracy (82.20%) and specificity (82.17%) among all models. Although it offers reasonable precision (82.77%) and recall (82.23%), it underperforms compared to ensemble methods. However, it is simple and interpretable, making it suitable for scenarios where explainability is prioritized over accuracy.
- **Random Forest:** The Random Forest model demonstrates strong performance with an accuracy of 86.37% and the highest specificity (88.35%), indicating excellent identification of non-subscribers. It also performs well in precision (85.40%) and recall (84.31%). This model is robust and effective, making it a reliable choice for classification tasks.
- **Support Vector Machine (SVM):** SVM achieves an accuracy of 85.12% with good balance between precision (83.83%) and recall (82.42%). It shows strong specificity (87.72%), making it particularly effective in minimizing false positives. While not the top performer overall, it is a competitive model, especially in binary classification tasks with balanced classes.
- **AdaBoost:** AdaBoost delivers the highest performance across most metrics: accuracy (86.51%), precision (86.14%), recall (85.35%), and F1-Score (85.74%). Its specificity (87.63%) is also very high. This balanced performance across all metrics makes AdaBoost the strongest overall model among those evaluated.

Conclusion and Best Model

Based on the performance metrics:

- **Best Overall Model: AdaBoost** is the top-performing model, with the highest accuracy, precision, recall, and F1-Score. It provides excellent balance in identifying both subscribers and non-subscribers.
- **Alternative Choice: Random Forest** is a strong alternative, especially if higher specificity is desired or if an ensemble model with easier interpretability is preferred.
- **Use Cases for SVM and Decision Tree:** SVM is effective when minimizing false positives is critical, while Decision Tree is best when simplicity and transparency are more important than predictive power.

Thus, **AdaBoost** is recommended as the best model for predicting term deposit subscriptions in this study.